

Day 10 — Live coding: from flows to a portfolio

Day 9 built one use case end to end. Today we build **nothing mechanically new** — no new step types, no new action anatomy. Instead we grow the assistant from six flows to nine and watch a different class of problem appear: flows *competing*. Every hard thing in this session is a judgement call, not a syntax question: what to name a flow, how to describe it, what the LLM gets to see, and where state lives. If the YAML feels easy today, that is the point — the YAML *is* easy now; the architecture is what's left.

You work in this lesson's `starting-point/` folder — the project exactly as Day 9 left it. The finished state ships alongside it in `end-result/` (`lumera-assistant/` , the grown `lumera-fastapi-server/` , and the `Makefile`); reach for it when a step says to copy a file in, or to compare against when you're done.

REQUIREMENTS — the project as Day 9 left it, `RASA_LICENSE` and `OPENAI_API_KEY` exported, `CORE_BANKING_URL=http://localhost:8000` and `CORE_BANKING_TOKEN=test_token` in any terminal that runs `rasa` or the mock bank (the `Makefile` sets the latter two for the processes it starts). **One new install:** section 6 uses classic-NLU components, which are an optional extra of Rasa Pro — `starting-point/`'s `requirements.txt` already says `rasa-pro[nlu]>=3.17` , so a fresh `make setup` covers it. (On Apple Silicon the extra pulls in `tensorflow-metal` , which is currently incompatible with the TensorFlow resolved next to it; the `Makefile` removes it after install — we use no TensorFlow components, our classifier is scikit-learn.)

1. Three new flows — the work is in the words

The portfolio grows by a **card cluster** — `block_card` and `replace_card` — plus `list_transactions` . Notice what we just did: we created two pairs of *near-neighbors*. Block and replace are both "card things"; transactions and the existing balance lookup are both "account things". A customer's message can plausibly mean either member of a pair. That collision is today's central subject, and we planted it on purpose.

The mechanics are all known. Each new flow uses the Day 8 action pattern (HTTP to the mock bank, `SlotSet` out) and the Day 9 trust decisions (confirmation gate on the destructive operation). The three actions contain nothing Day 8 didn't teach, so we won't show them in this guide for brevity — Either use the starting point or, if you're continuing from the last live coding, copy the files `lumera-assistant/actions/block_card.py` , `lumera-assistant/actions/replace_card.py` and `lumera-assistant/actions/fetch_transactions.py` from this lesson's `end-result/` into your project. The three mock endpoints they call (`POST /v1/cards/block` , `POST /v1/cards/replace` , `GET /v1/accounts/{account_type}/transactions`) are already part of today's mock bank server, which you run rather than edit. Today's session minutes go to the YAML — and mostly to the *words* in it.

You carried Day 9's `data/flows.yml` forward — six flows under one `flows:` key. The three new flows **append** to that same file: paste each one under the existing `flows:` key, alongside the flows already there. That is why the snippets below start at the flow name (`block_card:` , two-space indent)

with no `flows:` line of their own — they are fragments to add, not files to create. (Section 2 splits this monolith into per-flow files; until then, everything lives in `data/flows.yml`.)

Here is `block_card`, first version:

```
block_card:
  name: block a card
  description: Handle problems with the customer's card.
  steps:
    - call: authenticate_user
    - noop: true
    - next:
      - if: not slots.authenticated
        then:
          - action: utter_auth_failed
            next: END
        - else: ask_card
    - collect: card_type
    - id: ask_card
    - utter: utter_ask_card_to_block
    - description: "the card to block: debit or credit"
    - collect: block_confirmed
    - description: whether the customer confirms blocking the card
    - ask_before_filling: true
    - next:
      - if: slots.block_confirmed
        then:
          - action: action_block_card
            next:
              - if: slots.card_block_ok
                then: card_blocked
              - else:
                - action: utter_card_block_failed
                  next: END
        - else:
          - action: utter_card_block_declined
            next: END
    - action: utter_card_blocked
    - id: card_blocked
```

Three deliberate decisions in there:

- **call: authenticate_user is the first step.** Blocking a card is destructive, so it gets the same gatekeeper the transfer got on Day 9 — written once, reused. This is the payoff of the callable-only subflow: the second sensitive operation costs one line.
- **The confirmation gate** (`ask_before_filling: true` on `block_confirmed`) — blocking is irreversible from chat, so consent is collected fresh, every time, exactly like the transfer recap.
- **utter: utter_ask_card_to_block** — a `collect` step normally asks via `utter_ask_<slot_name>`; the `utter:` property lets us reuse the *same* `card_type` slot that `card_fees_info` already owns while asking a question that fits this flow ("Which card do you need to block?"). One slot, per-flow phrasing.

And the description? "*Handle problems with the customer's card.*" — we'll regret that in section 3, on purpose. (As always, the `end-result/` folder holds the *end state* — the corrected descriptions and the section-2 layout. To live the journey, type the versions shown here and fix them when we do.)

Now `replace_card`, appended right after it under the same `flows:` key:

```
replace_card:
  name: replace a card
  description: Help the customer with a card issue.
  steps:
    - collect: card_type
      utter: utter_ask_card_to_replace
      description: "the card to replace: debit or credit"
    - collect: replace_confirmed
      description: whether the customer confirms ordering the replacement card
      ask_before_filling: true
      next:
        - if: slots.replace_confirmed
          then:
            - action: action_replace_card
              next:
                - if: slots.card_replace_ok
                  then: card_replaced
                - else:
                    - action: utter_card_replace_failed
                      next: END
          - else:
            - action: utter_card_replace_declined
              next: END
    - action: utter_card_replacement_ordered
      id: card_replaced
```

Same skeleton — with no `call: authenticate_user`. That is a decision, not an omission: a replacement card ships to the registered address, so the damage a stranger can do by ordering one is small, and we chose not to tax the customer with a code for it. Blocking, by contrast, kills a working card instantly. *Per-flow trust decisions* are portfolio design: the two flows look like twins and deliberately differ in exactly one structural way. (Your bank's security team may decide differently — the point is that it is a per-flow decision, made consciously.)

`list_transactions` is `check_balance`'s shape with a different backend read — same `account_type` slot, fetch action, status-flag branch, and a response that prints the rows the action formatted into a `transactions_list` slot. Its first description: "*Show the customer what is in their account.*" Also regrettable, also on purpose — and, like the other two, appended under the existing `flows:` key:

```

list_transactions:
  name: list recent transactions
  description: Show the customer what is in their account.
  steps:
    - collect: account_type
      description: "the account whose transactions to list: checking or savings"
    - action: action_fetch_transactions
      next:
        - if: slots.transactions_fetch_ok
          then: show_transactions
        - else:
          - action: utter_transactions_service_down
            next: END
    - action: utter_recent_transactions
  id: show_transactions

```

The domain files hold the new slots — the two confirmation booleans, the `controlled` status flags, `card_replacement_eta`, `transactions_list` — plus the buttoned asks and outcome responses. All Day 7–8 material, three new files. `domain/block_card.yml`:

```

version: "3.1"

slots:
  block_confirmed:
    type: bool
    mappings:
      - type: from_llm
  card_block_ok:
    type: bool
    mappings:
      - type: controlled

responses:
  utter_ask_card_to_block:
    - text: "Which card do you need to block?"
      buttons:
        - title: "Debit card"
          payload: "/SetSlots(card_type=debit)"
        - title: "Credit card"
          payload: "/SetSlots(card_type=credit)"

  utter_ask_block_confirmed:
    - text: "I'm about to block your {card_type} card immediately – it can no longer be
      used after this. Shall I proceed?"
      buttons:
        - title: "Yes, block it"
          payload: "/SetSlots(block_confirmed=true)"
        - title: "No, don't"
          payload: "/SetSlots(block_confirmed=false)"

  utter_card_blocked:
    - text: "Done – your {card_type} card is now blocked. If you also need a
      replacement, just ask."

  utter_card_block_failed:
    - text: "I couldn't reach the card system, so the card has NOT been blocked. Please
      call the emergency number on our website if this is urgent."

  utter_card_block_declined:
    - text: "Understood – your {card_type} card stays active."

actions:
  - action_block_card

```

domain/replace_card.yml:

```

version: "3.1"

slots:
  replace_confirmed:
    type: bool
    mappings:
      - type: from_llm
  card_replace_ok:
    type: bool
    mappings:
      - type: controlled
  card_replacement_eta:
    type: text
    mappings:
      - type: controlled

responses:
  utter_ask_card_to_replace:
    - text: "Which card should I order a replacement for?"
      buttons:
        - title: "Debit card"
          payload: "/SetSlots(card_type=debit)"
        - title: "Credit card"
          payload: "/SetSlots(card_type=credit)"

  utter_ask_replace_confirmed:
    - text: "I'll order a replacement {card_type} card, delivered to your registered address. Shall I proceed?"
      buttons:
        - title: "Yes, order it"
          payload: "/SetSlots(replace_confirmed=true)"
        - title: "No, don't"
          payload: "/SetSlots(replace_confirmed=false)"

  utter_card_replacement_ordered:
    - text: "Done – your replacement {card_type} card is on its way and should arrive within {card_replacement_eta}."

  utter_card_replace_failed:
    - text: "I couldn't reach the card system, so no replacement has been ordered. Please try again later."

  utter_card_replace_declined:
    - text: "Understood – I won't order a replacement."

actions:
  - action_replace_card

```

domain/list_transactions.yml:

```

version: "3.1"

slots:
  transactions_list:
    type: text
    mappings:
      - type: controlled
  transactions_fetch_ok:
    type: bool
    mappings:
      - type: controlled

responses:
  utter_recent_transactions:
    - text: "Here are the latest movements on your {account_type}
account:\n{transactions_list}"

  utter_transactions_service_down:
    - text: "We're sorry, the transactions service is unavailable right now. Please try
again later."

actions:
  - action_fetch_transactions

```

Notice what is *not* declared here: `card_type`. Both card flows collect it, but it already belongs to Day 7's `card_fees_info.yml` — slots are conversation-wide, and the per-flow `utter:` phrasing is what makes the shared slot feel native to each flow.

Restart the mock bank — run `make run-fastapi-server` (the `starting-point/` server already carries today's three new endpoints) — then train and try one:

```

user: show me my recent transactions bot: Which account would you like to check? user:
checking bot: Here are the latest movements on your checking account: 2026-06-27 Salary —
Rossi & Partners +€2450.00 2026-06-28 Supermarket Esse-Piu -€74.30 2026-06-30 Card
payment — Trattoria da Peppe -€38.50 2026-07-01 Utility bill — electricity -€61.20 2026-07-03
ATM withdrawal -€100.00

```

(One nicety in the mock: executed transfers are appended to the checking list — send money and it shows up here, like the Day 9 balance deduction.)

2. Nine flows want a filesystem — one flow per file

`data/flows.yml` is now nine flows long, and the fortieth flow is coming. Before it does, we impose the at-scale layout: **one flow per file, grouped per topic** — and we retire `flows.yml`:

```
data/flows/
  accounts/      check_balance.yml, list_transactions.yml, download_statement.yml
  cards/         block_card.yml, replace_card.yml, card_fees_info.yml
  transfers/     transfer_money.yml
  appointments/  book_appointment.yml
  shared/        authenticate_user.yml
```

Honesty first: the *domain* has been split per topic since Day 7 — today just completes the same discipline on the flows side. Rasa reads everything under `data/` recursively, so the move is pure file surgery: cut each `flows:` entry into its own file (each file starts with its own `flows:` key), delete the monolith, and prove it was free:

```
rasa data validate
rasa train
```

Same assistant, zero behavior change. What we bought is the convention that scales: **names that grep**. The flow `block_card` lives in `block_card.yml`, asks via `utter_ask_...` responses named after its slots, is implemented by `action_block_card` — so `grep -r block_card` answers "what happens when a customer blocks a card?" across YAML, domain, and Python in one shot. When the portfolio hits forty flows, that convention *is* the documentation.

3. Steering: flow retrieval on, and descriptions become contrasts

Open `config.yml`. Since Day 7 it has carried:

```
flow_retrieval:
  active: false
```

We switched retrieval off when we had three flows and every token of prompt was easy to reason about. **Delete those two lines** — at nine flows and growing, the line earns its keep. What it does: at training time, every flow's description is embedded into a vector; at runtime the user message is embedded too, and only the *most similar* flows are put into the LLM's prompt. The costs, honestly: training now makes embeddings calls (your `OPENAI_API_KEY` is used at train time), and each turn embeds the incoming message. What it buys: the prompt no longer grows linearly with the portfolio — with hundreds of flows, the LLM still reads only a shortlist.

At our size the default shortlist (top 20) would include *everything*, so to actually **see** retrieval work, temporarily make it aggressive:

```
flow_retrieval:
  num_flows: 4
```

Retrain, and run the server with the command generator's prompt made visible:

```
LOG_LEVEL_LLM_COMMAND_GENERATOR=INFO rasa inspect
```


Type an ambiguous message — *"something is wrong with my card"* — and read the terminal. The rendered prompt contains a JSON list of candidate flows, and it has exactly four entries:

```
{"flows": [  
  {"name":"card_fees_info","description":"Explain the yearly fees of Banca Lumera payment cards.", ...},  
  {"name":"replace_card","description":"Help the customer with a card issue.", ...},  
  {"name":"block_card","description":"Handle problems with the customer's card.", ...},  
  {"name":"check_balance","description":"Look up the current balance of one of the customer's Banca Lumera accounts.", ...}  
]}
```

Two things to read off this, and the first answers a question this team has asked before:

- **Retrieval cannot pick a flow.** There is no confidence threshold to configure, and no path from "nearest vector" to "start the flow". Retrieval only decides *who is in the room*; the command generator — the LLM — still makes the decision, and you can watch it do so in the very next log line. If you were hoping to route on vector similarity alone and skip the LLM: this component will never do that. (Section 6 shows the mechanism that *does* skip the LLM.)
- **authenticate_user is not in the list — and never will be.** Its `if: False` guard from Day 9 excludes it from the prompt entirely, at any `num_flows`. A flow the model cannot see is a flow the model cannot start and no prompt injection can reach: guards are a *security* boundary, not just a routing convenience. (Don't be tempted to slap `if: slots.authenticated` on `block_card` itself, though — a guarded flow is invisible to *fresh* users too, so nobody could ever ask to block a card. The Day 9 shape is the right one: the flow stays startable, and the `call` gates the sensitive part *inside*.)

Now the collision we planted. The same log shows what the LLM did with our four candidates:

```
action_list: "disambiguate flows replace_card block_card block_card"
```

And in the chat:

user: something is wrong with my card **bot:** I can help, but I need more information. Which of these would you like to do: block a card, block a card or replace a card?

The engine did the right thing — faced with a message that genuinely fits several flows, it refused to guess and asked. This is the built-in **clarification pattern** doing its job, and we will *not* customize it (patterns get their own day). But look closer at the quality of the question: *"block a card"* is offered **twice**. Our two descriptions — *"Handle problems with the customer's card"* / *"Help the customer with a card issue"* — gave the model nothing to separate the flows with, and the mess shows up in the clarification itself.

The fix is upstream, in the words. **Write descriptions as contrasts** — say what the flow is *not* for whenever a neighbor could confuse:

```
block_card:
  description: >
    Immediately block one of the customer's Banca Lumera cards that is lost,
    stolen or compromised, so it can no longer be used. Does not order a
    replacement card.

replace_card:
  description: >
    Order a replacement for one of the customer's Banca Lumera cards that is
    damaged, expired or already blocked. Does not block an active card.
```

And the account pair:

```
check_balance:
  description: >
    Look up the current balance of one of the customer's Banca Lumera
    accounts. Does not list past transactions.

list_transactions:
  description: >
    List the most recent transactions on one of the customer's Banca Lumera
    accounts. Does not show the current account balance.
```

The "Does not..." sentences work twice: they pull the two descriptions' embedding neighborhoods apart (retrieval), and they hand the LLM the exact boundary (selection). Retrain and replay:

user: something is wrong with my card **bot:** I can help, but I need more information. Which of these would you like to do: block a card, card fees information or replace a card?

Still a clarification — *correctly*: that message carries no signal about which card operation is meant, and asking is the honest answer. But the question is now well-formed: three real, distinct options, no duplicates. And messages that *do* carry signal now route without a question:

user: I lost my card, someone might use it **bot:** For your security, please enter your one-time code. (→ *block_card*, straight through its auth gate)

user: my card is damaged, I need a new one **bot:** Which card should I order a replacement for? (→ *replace_card* — and note: no code asked. The asymmetry from section 1, visible.)

One honesty note: a strong LLM often routes the *obvious* messages correctly even with sloppy descriptions — don't let that lull you. You write contrasts for the boundary cases, for the quality of the clarifications, and for retrieval at a portfolio size where the model no longer sees every flow. Descriptions are cheap; review them **as a set**, hunting overlaps the way you review API routes for collisions.

Before moving on, remove the `num_flows: 4` line — it was a demo device, not architecture. Retrieval stays **on** with its defaults from here forward.

4. A digression, handled by machinery you already own

Mid-task topic switches are how humans talk. Start a transfer, and halfway through, ask something else:

user: send 50 euros to Marco Bassi **bot:** For your security, please enter your one-time code.
user: 123456 **bot:** What is the recipient's IBAN? **user:** IT12A0300203280123456789012 **bot:** Sorry, it seems the value you provided `50 euros` is not a valid number. Please provide a valid number in your response. **bot:** How much would you like to transfer, in euros? **user:** wait, what is my balance? **bot:** Which account would you like to check? **user:** the checking one **bot:** Your checking account balance is €1250.0. **bot:** Would you like to continue with transfer money? **user:** yes **bot:** How much would you like to transfer, in euros? **user:** 50 **bot:** Please confirm: transfer €50.0 to Marco Bassi, IBAN IT12A0300203280123456789012. Shall I proceed?

(The "not a valid number" exchange in the middle is the float slot type-rejecting the literal text "50 euros", exactly as on Day 9 — the free validation line doing its job before the digression even starts.)

Read what happened at *"wait, what is my balance?"*: the command generator started `check_balance`, which was **pushed on top of** the paused `transfer_money`; it ran to its end and popped; then a built-in pattern (`pattern_continue_interrupted`) offered the transfer back — with the recipient, IBAN and everything else still filled. Watch it in the Inspector's stack pane: two flows stacked during the side-trip, exactly like Day 9's `call` — except **nobody wrote a `call`**. The stack is the engine's; digressions come free.

What is *not* free is making them pleasant, and that is portfolio design again: small single-job flows and honest descriptions make the side-trip cost the customer one question, not a re-entry of everything. A balance check interrupting a transfer is helpful (they may be checking they can afford it); design your flows so the interrupters people actually reach for are cheap. The repair patterns that fired today — clarification in section 3, `continue-interrupted` here — are built-in, named, and customizable; *when* to customize them is Day 11's question, not today's.

5. State scoping — one challenge per session

Day 9 closed on a teaser: ask for a second transfer and the code is asked again. Now it bites for real. Complete a transfer (code asked, fine), then:

user: now block my debit card please **bot:** For your security, please enter your one-time code.

Two challenges in one session. For a bank's own authenticated app this is bad UX — the customer proved who they are ninety seconds ago.

To fix it properly, learn the **scoping rules** — where slot values live and die:

1. **Flow-local (the default):** slots filled by `collect` or `set_slots` steps are **reset when their flow ends**. A transfer's amount should not leak into the next topic; the default is right.
2. **Persisted past the flow:** a flow can exempt specific collect-set slots from that reset by listing them in its `persisted_slots` property. (Only collect/ `set_slots` -filled slots belong there — listing an action-set slot is a training-time error.)
3. **Slots set by custom actions persist on their own.** No reset applies to them at flow end.
4. **Across sessions:** `session_config.carry_over_slots_to_new_session` in the domain decides what survives a session boundary — a separate axis we already carry in `domain/shared.yml`.

Rule 3 is the surprise, and it explains our bug precisely. `authenticated` is set by `action_verify_customer` — an action-set slot — so it **already survives** the end of `authenticate_user` and the transfer. The state is there; look at the Inspector's memory pane after the transfer and you'll see `authenticated: true` sitting in it. The second challenge happens because the `call` re-runs the child flow, and the child *unconditionally asks for the code* (its collect is ask-always, by Day 9's own design). The fix is therefore not persistence — the fix is teaching the gatekeeper to check before it challenges. One branch at the top of `data/flows/shared/authenticate_user.yml`:

```
flows:
  authenticate_user:
    if: False
    name: verify customer identity
    description: Verify the customer's identity with a one-time security code before a
sensitive operation.
    steps:
      - noop: true
      next:
        - if: slots.authenticated
          then: END
        - else: ask_code
      - collect: security_code
      id: ask_code
      description: the one-time security code the customer provides
      ask_before_filling: true
      - action: action_verify_customer
```

Already verified → the child ends immediately → the parent's `call` returns in the same turn. Retrain and replay the pair:

user: *(transfer completed, code asked once)* now block my debit card please **bot:** I'm about to block your debit card immediately — it can no longer be used after this. Shall I proceed?

Straight to the consent gate. One challenge per session — and notice that *every* current and future sensitive flow inherits the fix, because they all `call` the same gatekeeper.

Say the security counterweight out loud before anyone asks: session-long authentication is a **policy choice**, not a technical inevitability. Per-operation re-auth is a defensible policy for payments; if your security team wants it, the Day 9 behavior was already correct. If you adopt session-long auth, decide how it *expires*: a deliberate `set_slots: [authenticated: null]` wherever policy says so (a timeout action, a logout flow). And check the cross-session axis: `carry_over_slots_to_new_session: true` has been in `shared.yml` since Day 7 — for a real bank you would review whether `authenticated` may outlive the session at all. State that outlives its justification is a liability, not a convenience.

Where `persisted_slots` is the right tool: a collect-set slot that a sibling flow can honestly reuse. Our pair: you check your savings balance, then ask for the movements — should the assistant re-ask "which account?" It already knows. In `check_balance.yml`:

```
check_balance:
  persisted_slots:
    - account_type
```

Retrain:

user: check my savings balance **bot:** Your savings account balance is €2087.5. **user:** and show me the transactions on it **bot:** Here are the latest movements on your savings account: 2026-06-15 Transfer from checking +€200.00 2026-06-30 Interest credit +€3.40

No re-ask — but be precise about *why*, because we tested both versions and the transcript alone doesn't tell you. Watch the Inspector's memory pane at the moment the balance flow ends: **with** `persisted_slots`, `account_type: savings` is still sitting there; **without** it, the value is wiped at that instant (rule 1). In this particular conversation the un-persisted version *also* happened not to re-ask — the LLM re-derived "savings" from the previous message on its own. That is exactly the difference that matters: without persistence you are relying on the model's contextual goodwill, which varies with phrasing, distance and model version; with `persisted_slots` the value is *structurally* in memory, deterministically, for every flow that comes after. Today's whole theme in one slot: state you care about gets a declared scope — it is not left to inference. And that is the judgement to make flow by flow: *is this slot a working value (let it die) or session state (persist it, and write down why)?*

6. Extra — the NLU sidecar: starting flows without the LLM

As an extra exercise, let's see how we can use a classic NLU classifier to start flows without invoking the LLM at all.

Three pieces. First, classic NLU training data — a new `data/nlu.yml`:

```

nlu:
- intent: check_balance
  examples: |
    - what's my balance
    - show me my balance
    - how much money do I have
    - check my account balance
    - what is my current balance
    - how much is in my checking account
    - how much is in my savings account
    - balance please
    - can you check my balance
    - what's the balance on my account

- intent: list_transactions
  examples: |
    - show me my recent transactions
    - what are my latest transactions
    - list my account movements
    - what did I spend recently
    - show the last movements on my account
    - recent activity on my checking account
    - what transactions went through my savings account
    - show my transaction history
    - list my recent payments
    - what happened on my account lately

```

Why *two* intents when we only wire one? A classifier refuses to train on a single class — `rasa train` fails at the classifier's training node with one intent defined. Two is the floor; `list_transactions` also gives the classifier a realistic contrast for account-ish messages. Declare both in the domain — a new `domain/nlu.yml`:

```

version: "3.1"

intents:
- check_balance
- list_transactions

```

Second, the pipeline in `config.yml` grows a classic front end:

```

pipeline:
- name: WhitespaceTokenizer
- name: CountVectorsFeaturizer
- name: LogisticRegressionClassifier
- name: NLUCommandAdapter
- name: CompactLLMCommandGenerator
  llm:
    model_group: openai_llm

```

Tokenizer → featurizer → intent classifier is the smallest classic-NLU stack (scikit-learn logistic regression — fast, no deep learning); the `NLUCommandAdapter` is the bridge: it reads the predicted intent and, if some flow declares a matching trigger, issues that flow's start command itself. (These components are why `requirements.txt` gained the `[nlu]` extra.)

Third, the trigger, on the flow — in `check_balance.yml`:

```
check_balance:
  nlu_trigger:
    - intent:
        name: check_balance
        confidence_threshold: 0.75
```

The threshold is measured, not guessed. Run `rasa run --enable-api` and POST a few messages to `/model/parse` to see what the classifier actually says:

```
curl -X POST http://localhost:5005/model/parse -H "Content-Type: application/json" -d '{"text": "what is my balance"}'
```

message	predicted intent	confidence
"what is my balance"	check_balance	0.81
"show me my recent transactions"	list_transactions	0.83
"block my debit card"	list_transactions	0.58
"123456"	list_transactions	0.55

Real balance phrasings score ~0.80; everything else this tiny two-class model must still label lands near 0.55. So 0.75 separates cleanly — while a "safe-sounding" 0.9 would simply never fire, and the sidecar would be decoration. (Note the classifier happily labels "block my debit card" as `list_transactions` — a classic-NLU model *a/ways* answers with something from its intent list; the threshold is what keeps that nonsense from acting.)

Retrain and watch the logs (`LOG_LEVEL_NLU_COMMAND_ADAPTER=INFO` `LOG_LEVEL_LLM_COMMAND_GENERATOR=INFO`):

user: what's my balance **bot:** Which account would you like to check?

```
nlu_command_adapter.predict_commands {"commands":
  ["StartFlowCommand(flow='check_balance')"]}
```

...and **no prompt line follows** — the flow started with *zero* LLM calls on that turn. That is the command generator's `minimize_num_calls` parameter (default `true`): when the adapter already resolved the turn, the LLM invocation is skipped entirely. Now the next turn:

user: checking **bot:** Your checking account balance is €1250.0.

```
nlu_command_adapter.predict_commands {"commands": []}
llm_command_generator ... "action_list": "set slot account_type checking"
```

The adapter can start flows via intents and fill slots only from classic NLU signals (entities/intents) — it cannot fill our `from_llm` slot, so on this turn the LLM steps back in as the fallback. The two understanding systems interleave turn by turn, cheapest-first. Every other message in the assistant — cards, transfers, appointments — still takes the LLM path untouched.

One honest cost: try *"check my savings balance"*. The intent triggers, the LLM is skipped — and the word **"savings" is dropped**, because the component that would have extracted it never ran; the flow re-asks which account. On the pure LLM path that message filled the slot in the same turn. That is the trade `minimize_num_calls` makes: on trigger turns you pay zero LLM tokens and lose the LLM's slot extraction. If the messages that hit your triggers routinely carry slot values, set `minimize_num_calls: false` on the command generator — both components then run every turn (the adapter's commands win any conflict) and you buy the extraction back at the price of the call you were saving.

One security caveat, non-negotiable: flow guards do **not** gate NLU-triggered starts. Guards restrain the command generator; `call`, `link`, and `nlu_trigger` all walk past them. A flow that is guard-gated for security and also carries an `nlu_trigger` is an authentication bypass waiting to be found. The rule: never put `nlu_trigger` on a flow whose guard you rely on. `check_balance` is safe on both counts — it has no guard, and it exposes nothing destructive. If you ever need a triggered flow to be safe under *every* activation path, do the check *inside* the flow, the way our sensitive flows already `call` the gatekeeper.

Two closing notes on the bigger coexistence picture, told rather than typed. If you need to run a whole *legacy NLU assistant* — intents, rules, stories, years of tuning — beside CALM in one project, Rasa's coexistence setup does that with a **router** in front (an `IntentBasedRouter` routing on the classifier's intent at zero extra cost, or an `LLMBasedRouter` spending a call to route free-form phrasings); the decision is recorded per conversation in a slot and sticks, so a process finishes on the side that started it. That is a migration architecture, and it stays out of our snapshot. And above single-assistant scale, decomposition has one more axis — flows can delegate to sub agents, and assistants can talk to other assistants — which this course treats later; today, one assistant owns the whole portfolio.

7. Where we landed

```
lumera-assistant/
├─ actions/
│  └─ ... (Day 8–9 actions, unchanged)
│     └─ block_card.py, replace_card.py, fetch_transactions.py  # NEW – Day 8 pattern
├─ config.yml  # retrieval ON, classic-NLU sidecar in the pipeline
├─ data/
│  └─ nlu.yml  # NEW – two intents for the sidecar
│     └─ flows/  # NEW layout – one flow per file, per topic
│        └─ accounts/  cards/  transfers/  appointments/  shared/
└─ domain/  # + block_card, replace_card, list_transactions, nlu
lumera-fastapi-server/  # + /v1/cards/block, /v1/cards/replace,
                        #   /v1/accounts/{type}/transactions
```

Nine flows, and the day's actual deliverables are invisible in a file tree: descriptions written as **contrasts**, a layout whose **names grep**, guards read as a **security boundary**, state given a **scope on purpose**, and an LLM that is no longer the only — or even the first — way a flow starts. Nothing today was a new mechanism; all of it was architecture.

One thing we did *not* do: when the clarification fired, when the digression pattern offered the transfer back — we watched built-in repair behavior save us, and left it stock. Next session we stop being polite to the assistant: we break the transfer six ways on purpose, name the pattern that catches each fall, and then freeze the whole gauntlet into tests.